

COSC 1030 Physical Computing Project 3

This project will be an introduction to a sensor that can interface with the Raspberry Pi to report temperature and humidity. We will also be learning about virtual environments and continuing to use the multiprocessing package in python to manage multiple processes running simultaneously.

Hardware needed:

- Breadboard
- LED and resistor
- DHT22 temperature humidity sensor
- Button
- Jumper wires

Part 1: Learning about the dht22 sensor, virtual environments, and testing your sensor

1. The Circuit Setup

The dht22 sensor looks like:



Wiring Instructions:

- **VCC (+):** Connect to **3.3V** (Physical Pin 1 on the Pi). - this is the pin marked “+” on the dht22
- **Data (out):** Connect to **GPIO 4** (Physical Pin 7 on the Pi). - this is the middle pin on the dht22 that has “out”
- **GND (-):** Connect to a **Ground** (Physical Pin 9 on the Pi). - this is the pin marked “-” on the dht22

(Note that the dht22 sensor has a built in resistor so we don't need a resistor in this circuit. Also note that VCC (+) could be connected to any 3.3V pin; Data (out) could be connected to any

standard GPIO pin GPIO 4, 17, 18, 22, 23, 24, 25, 27 – just make sure that if you are using a different GPIO, that you change the line of code that creates the sensor object and connects it with GPIO 4; GND can be connected to any Ground)

2. Setting Up the Virtual Environment

In this project we are going to set up a virtual environment. In modern Python development, we don't just install things anywhere. We create a "Virtual Environment" (venv) to keep our project libraries separate from the rest of the computer. This process makes sure that the python code that depends on other libraries (and maybe a particular version of a library) won't be broken if the library is updated for the entire system. If you want to know more about virtual environments you should ask your favorite AI engine to tell you more.

Commands for the Terminal:

1. **Navigate to your project folder:**
`cd ~/Desktop/COSC1030/Hardware/dht22`
2. **Create the environment:**
`python3 -m venv dht_env`
3. **Activate it:**
`source dht_env/bin/activate`
(You should now see *(dht_env)* at the start of your command line!)
4. **Install the sensor library:**
`pip install adafruit-circuitpython-dht`
5. **Install a helper library for Raspberry Pi hardware:**
`sudo apt-get install libgpiod2`

If you have any trouble with any of the above steps, this is an excellent time to use your favorite AI engine to troubleshoot.

NOTE: you will need to run all the code for this project from within the virtual environment, so you will need to do step 3 to get to the prompt that starts with *(dht_env)* before using python3 to run any code.

NOTE: To leave the virtual environment, at the command line type:

`deactivate`

3. Starter Code: `sensor_test.py`

```

import time
import board
import adafruit_dht

sensor = adafruit_dht.DHT22(board.D4)

while True:
    try:
        temp_c = sensor.temperature
        humidity = sensor.humidity

        print(f"Temperature: {temp_c:.1f}C | Humidity: {humidity:.1f}%")
    except RuntimeError:
        print(f"Sensor Glitch")

    time.sleep(2.0) # no need to constantly be taking readings
~

```

Note the line:

```
sensor = adafruit_dht.DHT22(board.D4)
```

This line creates a sensor object from the adafruit library that interacts with the data from the sensor. The 'D4' hooks the object to GPIO 4. If you used a different GPIO, you should change this to whatever you used.

At this point you should be able to run the code. Note in this code the try-except construct. This is something that we have not seen, but is very useful here. The dht22 sensor is notoriously glitchy. When we try to read from the sensor (in the line "temp_c = sensor.temperature" we might not end up with a numeric value that can be printed out. Since the print is inside the 'try' if it fails, instead of crashing the program, the program just does what is in the 'except' clause and prints out an error message. There is no need to read from the sensor as often as possible, so the time.sleep(2.0) delays each read by 2 seconds. Because of the 'while True: ' loop, you will need to hit CTRL-C to quit the program.

The result should look something like this:

```

Sensor Glitch
Temperature: 23.1C | Humidity: 19.8%
Temperature: 23.1C | Humidity: 19.8%
Temperature: 23.1C | Humidity: 19.8%
Temperature: 23.1C | Humidity: 19.7%
Temperature: 23.1C | Humidity: 19.7%
Temperature: 23.1C | Humidity: 19.8%
Temperature: 23.1C | Humidity: 19.8%
Temperature: 23.1C | Humidity: 19.7%
Sensor Glitch
Temperature: 23.1C | Humidity: 19.7%
Temperature: 23.1C | Humidity: 19.7%
Temperature: 23.1C | Humidity: 19.7%
Temperature: 23.1C | Humidity: 19.7%
Temperature: 23.1C | Humidity: 19.7%
Temperature: 23.1C | Humidity: 19.7%
Temperature: 23.1C | Humidity: 19.7%
Temperature: 23.1C | Humidity: 19.8%
Temperature: 23.1C | Humidity: 19.8%
Temperature: 23.1C | Humidity: 19.7%
Temperature: 23.1C | Humidity: 19.8%
Temperature: 23.1C | Humidity: 19.6%
Temperature: 23.2C | Humidity: 19.7%
Temperature: 23.2C | Humidity: 19.7%

```

NOTE: If this program is not working, double check your wiring. BUT these sensors are also notorious for failing completely. If your sensor does not appear to be working, please come get another one.

NOTE: You can change the temperature by gently holding onto the sensor. You should see the temperature and humidity go up. When you let go of the sensor, it should go back down. These temperatures are in Celsius by default, but you can easily do the calculation to convert to Fahrenheit if you want to!

There is another potential problem with this sensor. This actually caused me a lot of headaches, so I'm going to pass along what I've learned and hope to save you some time. It turns out that the ada_fruit library launches a process that reads from the sensor. When you hit Ctrl+C to end the program, the python script will end, but that background "zombie" process that is tying up the GPIO pin may still be running. The way to avoid this is to actually use another "try" with a "finally" clause. The "finally" clause will run before the python script quits under all

circumstances. So if we put code in the finally that will stop the background process, we can avoid this problem. Here's the modified code:

```
import time
import board
import adafruit_dht

sensor = adafruit_dht.DHT22(board.D18) # Data on GPIO18

try:
    while True:
        try:
            temp_c = sensor.temperature
            humidity = sensor.humidity

            print(f"Temperature: {temp_c:.1f}C | Humidity: {humidity:.1f}%")
        except RuntimeError:
            print(f"Sensor Glitch")

        time.sleep(2.0) # no need to constantly be taking readings
finally: #This code will run before python script ends
    print("After stopping with CTRL-C")
    sensor.exit()
```

Note the use of `sensor.exit()` here to release the pin and prevent the zombie process.

Part 2: The Visual Feedback (Adding the LED)

NOTE: Make sure you are still in your virtual environment before you start this task since we will be installing more software packages. (You will need to do `source dht_env/bin/activate`)

1. The Circuit Expansion

We will add a single LED to the breadboard. Since we are using a Raspberry Pi, we must use a **current-limiting resistor** (usually $\$220\Omega$ to $\$330\Omega$) to prevent the LED from burning out or damaging the Pi.

Wiring Instructions:

- **LED Anode (Long Leg):** Connect to **GPIO 22** (Physical Pin 15).
- **LED Cathode (Short Leg):** Connect to one side of the **Resistor**.

- **Resistor other side:** Connect to **Ground** (Physical Pin 6 or any GND pin).

Recall that you can test to see if you have the led connected properly by attaching the anode end to physical pin 2, which is connected to 5v so the light should automatically light up.

2. Update the Virtual Environment

We need to install the `gpiozero` library into this virtual environment, which makes controlling LEDs incredibly simple as we have seen before.

Command for the Terminal:

(Ensure your venv is active first!)

```
pip install gpiozero lgpio
```

Now add some code to turn the light on if some temperature threshold is exceeded and turn the light off if the temperature falls below the threshold. You will want to add the line:

```
from gpiozero import LED
```

To the `import` section of the code and the line

```
led = LED(22)
```

To create an LED object that is connected to GPIO 22

The code that turns on the led looks like:

```
led.on()
```

The code that turns off the led looks like

```
led.off()
```

Figure out how to add the code that will turn on when the temperature goes above the threshold and turn off when the temperature goes below the threshold. Test your code to make sure it is running correctly. Note that the LED object can also end up with a zombie process, so in the 'finally' part of the code put:

```
led.close()
```

To stop this process and release the GPIO pin controlling the LED.

TURN IN: a copy of your code and a video tour.

Question: In [Physical Computing Project 2](#) we were able to make the LED blink out different messages. What if we wanted the LED to blink a pattern when the temperature goes above the threshold instead of just turning on and off? Can we add this functionality to the program that you just wrote?

Part 3: Use the multiprocessing package to manage the LED and DHT22 sensor processes

The answer that you should have come up with is that if you set the led blinking with a while True loop, now the program is basically stuck in that loop forever and there is not an easy way to get it back. The answer to this is to go back to using the multiprocessing package to be able to manage different processes. Use what you have learned about the multiprocessing package to have a separate process that runs the blinking light that you can stop when needed.

TURN IN: A copy of your code and a video tour.

Part 4: Adding a button and having shared resources between processes

The scenario: You are designing a system that can monitor the temperature in either a greenhouse or a data center. The system should allow a temperature threshold to be toggled by pressing a button. If the temperature threshold is set for the greenhouse, pressing the button should set the threshold for the data center. Presumably the threshold for the greenhouse is higher than the temperature for the data center. If the temperature is higher than whatever the current threshold is set to, the LED should blink out a custom pattern.

The final goal of this part is to add a button to the project and code that will monitor the button. The press of the button will control the mode that the program is running in. The press of the button should change the threshold value that triggers the threshold value to change. Go ahead and set up the button circuit. I used GPIO 17 with the button, so the code that I'm going to show below will use GPIO 17 for the button (as well as GPIO 18 for the DHT22 and GPIO 22 for the LED).

To accomplish this task we will need to have

- A process that is monitoring the DHT22 sensor for the temperature value.
- A process that is monitoring the button that will toggle the threshold value that triggers the led blinking function.
- A process that is running the led blinking function if the temperature is above the threshold value.

Both the sensor process and the button process will need to have access to the variable that is storing the threshold value. Your gut instinct here might be to use a global variable that in a normal program would be visible to the entire python program. BUT when you use the multiprocessing package, each process essentially gets its own copy of the entire script including the global variables. Each process would then be accessing a different variable when it needed to use the threshold variable.

The solution to this problem is also provided by the multiprocessing package. We can create a shared object that all processes can use. The line of code that does this is going to look like this

```
shared_threshold = multiprocessing.Value('d', 23)
```

This shared_threshold object can be seen and modified by any of the processes that are created.

The value stored in this object (in this case it would be 23) can be accessed by:

```
shared_threshold.value
```

This code will also use a multiprocessing.Event() object that will allow the light blinking process to end gracefully instead of just terminating it and maybe leaving the light one. This is another form of a shared resource between processes. The sensor_loop has code that will detect when the blinking light needs to stop and can basically set a flag that the blink function can access. So instead of a 'while True' loop inside the blink function there will be a 'while not stop_event.is_set()'.

Since the code to accomplish this task is more complex than we have seen so far, I'm going to give you a lot of it and have you fill in the parts that are missing that are similar (or the same) as stuff we have done before. That way you won't have to write all of this code, but you can see how this kind of multiprocessing code is put together and hopefully learn about it as you finish the project. To get the starter code:

```
git clone https://github.com/charlottegruner/COSC1030_PhysicalComputing3
```

The parts of the code that you should add to this are marked with a **### TO DO** comment.

You are welcome to use AI to help debug and work through this project and to learn more about the lines of code in it. For your final project in this class you will be choosing another sensor or output device that can be integrated into the raspberry pi GPIOs, learning about it, and developing a project that uses the multiprocessing package.

TURN IN: Your code and a video.